Experiment - 10

Nikhil Kale
D15A/50

Aim - Explore and implement Autoencoders for image denoising

Dataset Source

Dataset used:
https://www.kaggle.com/datasets/hojjatk/mnist-dataset

Title: MNIST Handwritten Digit Dataset

Dataset Description

The MNIST dataset consists of grayscale images of handwritten digits.

- Total images: **70,000**
- Image size: **28×28 pixels**
- Classes: **10 digits (0–9)**

Characteristics:

- Clean dataset used for image processing tasks
- Suitable for deep learning models
- Used here for image denoising

Mathematical Formulation

Autoencoder

An autoencoder learns:

$$h = f(x)$$

$$\hat{x} = g(h)$$

Where:

- f = encoder

- g = decoder

Loss Function

Binary Cross-Entropy:

$$L = -\sum[x\log(\hat{x}) + (1-x)\log(1-\hat{x})]$$

Objective

Minimize reconstruction error:

$$\min ||x - \hat{x}||$$

Algorithm Limitations

- Requires large training data
- May reconstruct noise if overfitted
- Not suitable for highly complex noise patterns
- Training can be computationally expensive

Methodology / Workflow

1. Load MNIST dataset
2. Normalize images
3. Add artificial noise
4. Build autoencoder model
5. Train model on noisy images
6. Predict denoised images
7. Compare results

Workflow:

Original Image → Add Noise → Autoencoder → Reconstructed Image

Performance Analysis

Evaluation Metrics:

- Reconstruction Loss

- Visual quality comparison

Observations:

- Autoencoder successfully removes noise
- Output images closely resemble original images
- Loss decreases during training
- Slight blurring may occur

Hyperparameter Tuning

Parameters tuned:

- Number of convolution layers
- Filters (32, 64)
- Epochs (10)
- Batch size (128)

Observations:

- More layers improve reconstruction
- Too many layers may overfit
- Adam optimizer gives stable training

Code and Output -

[11] ✓ 0s

```
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense, Conv2D, MaxPooling2D, UpSampling2D
from tensorflow.keras.datasets import mnist
```

[12] ✓ 0s

```
(X_train, _), (X_test, _) = mnist.load_data()
```

[13] ✓ 0s

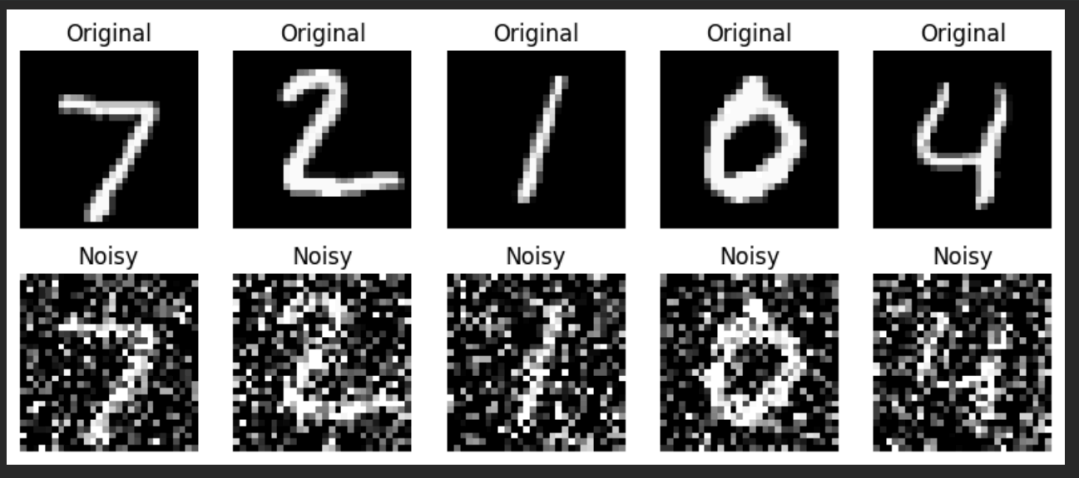
```
X_train = X_train / 255.0
X_test = X_test / 255.0

X_train = X_train.reshape(-1,28,28,1)
X_test = X_test.reshape(-1,28,28,1)
```

[14] ✓ 2s

```
noise_factor = 0.5

X_train_noisy = X_train + noise_factor * np.random.normal(loc=0.0, scale=1.0, size=X_train.shape)
X_test_noisy = X_test + noise_factor * np.random.normal(loc=0.0, scale=1.0, size=X_test.shape)

# Clip values
X_train_noisy = np.clip(X_train_noisy, 0., 1.)
X_test_noisy = np.clip(X_test_noisy, 0., 1.)
```

[15]
0s

```python
plt.figure(figsize=(10,4))
for i in range(5):
    plt.subplot(2,5,i+1)
    plt.imshow(X_test[i].reshape(28,28), cmap='gray')
    plt.title("Original")
    plt.axis('off')

    plt.subplot(2,5,i+6)
    plt.imshow(X_test_noisy[i].reshape(28,28), cmap='gray')
    plt.title("Noisy")
    plt.axis('off')
plt.show()
```



[16]
0s

```python
input_img = Input(shape=(28,28,1))

# Encoder
x = Conv2D(32, (3,3), activation='relu', padding='same')(input_img)
x = MaxPooling2D((2,2), padding='same')(x)

x = Conv2D(64, (3,3), activation='relu', padding='same')(x)
encoded = MaxPooling2D((2,2), padding='same')(x)

# Decoder
x = Conv2D(64, (3,3), activation='relu', padding='same')(encoded)
x = UpSampling2D((2,2))(x)

x = Conv2D(32, (3,3), activation='relu', padding='same')(x)
x = UpSampling2D((2,2))(x)

decoded = Conv2D(1, (3,3), activation='sigmoid', padding='same')(x)

autoencoder = Model(input_img, decoded)
```

[17]
0s

```python
autoencoder.compile(
    optimizer='adam',
    loss='binary_crossentropy'
)
```

[18]
29m

```
history = autoencoder.fit(
    X_train_noisy, X_train,
    epochs=10,
    batch_size=128,
    shuffle=True,
    validation_data=(X_test_noisy, X_test)
)
```

```
Epoch 1/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 167s 348ms/step - loss: 0.1616 - val_loss: 0.1146
Epoch 2/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 167s 355ms/step - loss: 0.1114 - val_loss: 0.1064
Epoch 3/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 161s 343ms/step - loss: 0.1056 - val_loss: 0.1028
Epoch 4/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 161s 342ms/step - loss: 0.1027 - val_loss: 0.1008
Epoch 5/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 202s 342ms/step - loss: 0.1006 - val_loss: 0.0990
Epoch 6/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 202s 342ms/step - loss: 0.0993 - val_loss: 0.0980
Epoch 7/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 165s 352ms/step - loss: 0.0984 - val_loss: 0.0973
Epoch 8/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 197s 342ms/step - loss: 0.0977 - val_loss: 0.0965
Epoch 9/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 164s 351ms/step - loss: 0.0970 - val_loss: 0.0963
Epoch 10/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 165s 351ms/step - loss: 0.0964 - val_loss: 0.0957
```

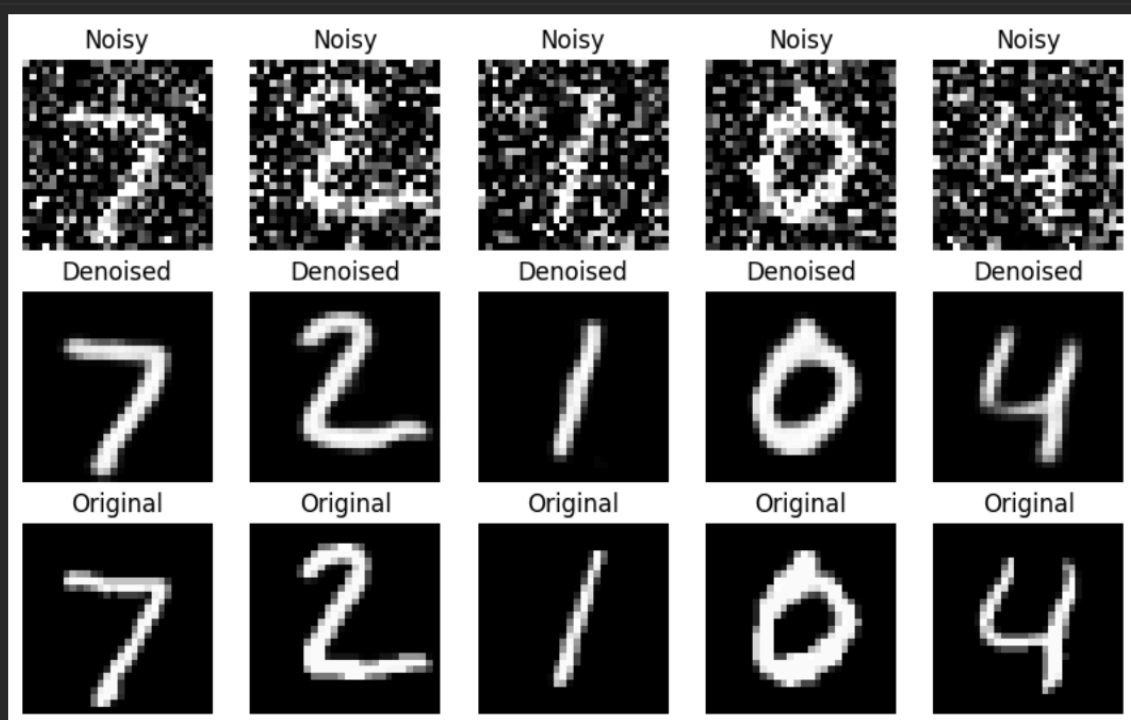
[19]
10s

```
decoded_imgs = autoencoder.predict(X_test_noisy)
```

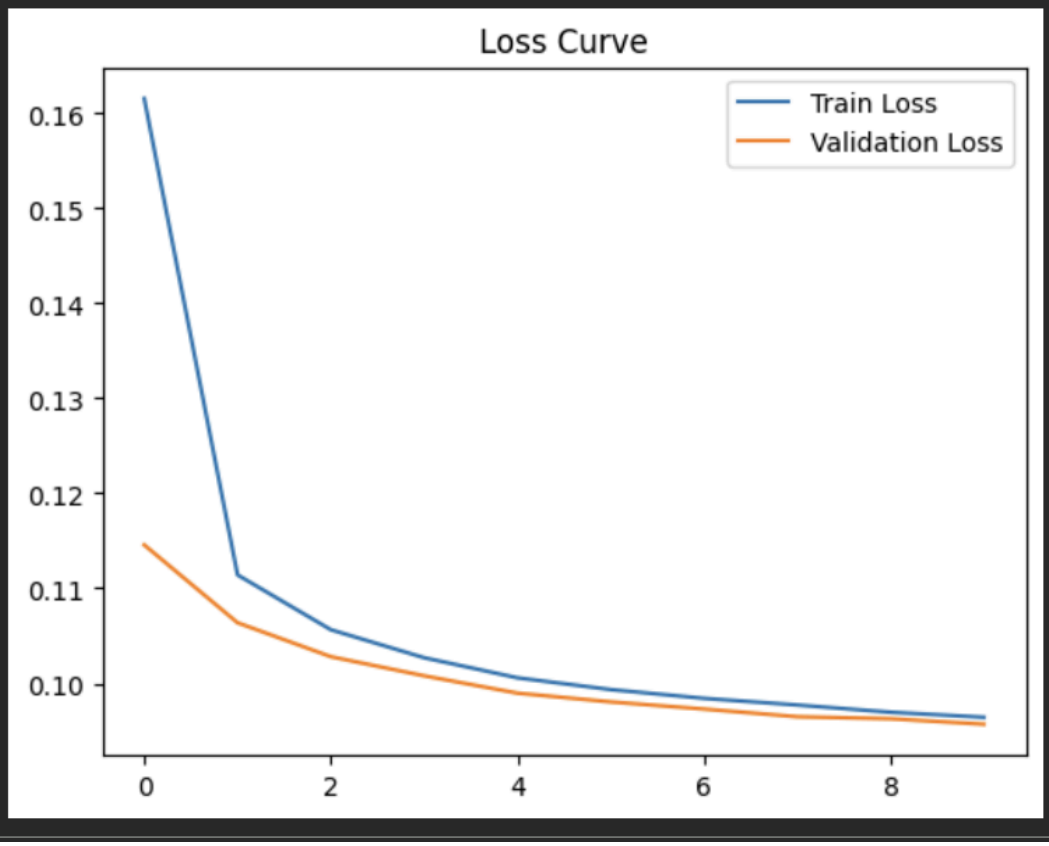
```
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 20ms/step
```

[20]
0s

```python
plt.figure(figsize=(10,6))

for i in range(5):
    # Noisy
    plt.subplot(3,5,i+1)
    plt.imshow(X_test_noisy[i].reshape(28,28), cmap='gray')
    plt.title("Noisy")
    plt.axis('off')

    # Denoised
    plt.subplot(3,5,i+6)
    plt.imshow(decoded_imgs[i].reshape(28,28), cmap='gray')
    plt.title("Denoised")
    plt.axis('off')

    # Original
    plt.subplot(3,5,i+11)
    plt.imshow(X_test[i].reshape(28,28), cmap='gray')
    plt.title("Original")
    plt.axis('off')

plt.show()
```



```
[21]
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Loss Curve")
plt.show()
```



Conclusion

In this experiment, an autoencoder was implemented for image denoising using the MNIST dataset. The model successfully learned to reconstruct clean images from noisy inputs, demonstrating the effectiveness of deep learning in image restoration tasks.